Trạng thái	Đã xong
Bắt đầu vào lúc	Thứ Bảy, 27 tháng 7 2024, 10:56 AM
Kết thúc lúc	Thứ Sáu, 23 tháng 8 2024, 3:49 PM
Thời gian thực hiện	27 Các ngày 4 giờ
Điểm	8,00/8,00
Điểm	**10,00** trên 10,00 (**100**%)

Câu hỏi **1**

Đúng

Đạt điểm 1,00 trên 1,00

In this question, you have to perform add **and delete on binary search tree**. Note that:

- When deleting a node which still have 2 children, **take the inorder successor** (smallest node of the right sub tree of that node) to replace it.

- When adding a node which has the same value as parent node, add it in the **left sub tree**.

Your task is to implement two functions: add and deleteNode. You could define one or more functions to achieve this task.

```
#include <iostream>
#include <string>
#include <sstream>
using namespace std;
#define SEPARATOR "#<ab@17943918#@>#"
template<class T>
class BinarySearchTree
{
public:
    class Node;
private:
    Node* root;
public:
    BinarySearchTree() : root(nullptr) {}
    ~BinarySearchTree()
    {
        // You have to delete all Nodes in BinaryTree. However in this task, you can ignore it.
    }

    //Helping function

    void add(T value){
        //TODO
    }

    void deleteNode(T value){
        //TODO
    }
    string inOrderRec(Node* root) {
        stringstream ss;
        if (root != nullptr) {
            ss << inOrderRec(root->pLeft);
            ss << root->value << " ";
            ss << inOrderRec(root->pRight);
        }
        return ss.str();
    }

    string inOrder(){
        return inOrderRec(this->root);
    }

    class Node
    {
    private:
        T value;
        Node* pLeft, * pRight;
        friend class BinarySearchTree<T>;
    public:
        Node(T value) : value(value), pLeft(NULL), pRight(NULL) {}
        ~Node() {}
    };
};
```

For example:

Test	Result

BinarySearchTree<int> bst;
bst.add(9);
bst.add(2);
bst.add(10);
bst.deleteNode(9);
cout << bst.inOrder();
	2 10

Test	Result

BinarySearchTree<int> bst;
bst.add(9);
bst.add(2);
bst.add(10);
bst.add(8);
cout << bst.inOrder()<<endl;
bst.add(11);
bst.deleteNode(9);
cout << bst.inOrder();
	2 8 9 10
	2 8 10 11

Answer: (penalty regime: 5, 10, 15, ... %)

Reset answer

```
// Add a value to the tree
    void add(T value) {
        root = addRec(root, value);
    }

    // Delete a value from the tree
    void deleteNode(T value) {
        root = deleteRec(root, value);
    }
Node* addRec(Node* node, T value) {
        if (node == nullptr) {
            return new Node(value);
        }
        if (value < node->value) {
            node->pLeft = addRec(node->pLeft, value);
        } else {
            node->pRight = addRec(node->pRight, value);
        }
        return node;
    }
    // Helper function to find the inorder successor
    Node* minValueNode(Node* node) {
        Node* current = node;
        while (current && current->pLeft != nullptr)
            current = current->pLeft;
        return current;
    }
    // Helper function to delete a value recursively
    Node* deleteRec(Node* root, T value) {
        if (root == nullptr) return root;

        if (value < root->value) {
            root->pLeft = deleteRec(root->pLeft, value);
        } else if (value > root->value) {
            root->pRight = deleteRec(root->pRight, value);
        } else {
            if (root->pLeft == nullptr) {
                Node* temp = root->pRight;
                delete root;
                return temp;
            } else if (root->pRight == nullptr) {
                Node* temp = root->pLeft;
                delete root;
                return temp;
            }

            Node* temp = minValueNode(root->pRight);
            root->value = temp->value;
            root->pRight = deleteRec(root->pRight, temp->value);
        }
        return root;
    }
```

	Test	Expected	Got	

✓	`BinarySearchTree<int> bst;`
	`bst.add(9);`
	`bst.add(2);`
	`bst.add(10);`
	`bst.deleteNode(9);`
	`cout << bst.inOrder();`
		`2 10`	`2 10`	✓

✓	`BinarySearchTree<int> bst;`
	`bst.add(9);`
	`bst.add(2);`
	`bst.add(10);`
	`bst.add(8);`
	`cout << bst.inOrder()<<endl;`
	`bst.add(11);`
	`bst.deleteNode(9);`
	`cout << bst.inOrder();`
		`2 8 9 10`
		`2 8 10 11`
			`2 8 9 10`
			`2 8 10 11`
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **2**

Đúng

Đạt điểm 1,00 trên 1,00

Class `BSTNode` is used to store a node in binary search tree, described on the following:

```
class BSTNode {
public:
    int val;
    BSTNode *left;
    BSTNode *right;
    BSTNode() {
        this->left = this->right = nullptr;
    }
    BSTNode(int val) {
        this->val = val;
        this->left = this->right = nullptr;
    }
    BSTNode(int val, BSTNode*& left, BSTNode*& right) {
        this->val = val;
        this->left = left;
        this->right = right;
    }
};
```

Where `val` is the value of node, `left` and `right` are the pointers to the left node and right node of it, respectively. If a repeated value is inserted to the tree, it will be inserted to the left subtree.

Also, a static method named `createBSTree` is used to create the binary search tree, by iterating the argument array left-to-right and repeatedly calling `addNode` method on the root node to insert the value into the correct position. For example:

```
int arr[] = {0, 10, 20, 30};
auto root = BSTNode::createBSTree(arr, arr + 4);
```

is equivalent to

```
auto root = new BSTNode(0);
root->addNode(10);
root->addNode(20);
root->addNode(30);
```

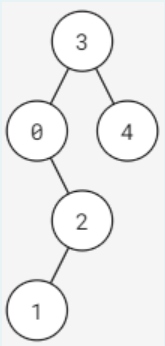
Request: Implement function:

```
vector<int> levelAlterTraverse(BSTNode* root);
```

Where `root` is the root node of given binary search tree (this tree has between 0 and 100000 elements). This function returns the values of the nodes in each level, alternating from going left-to-right and right-to-left..

Example:

Given a binary search tree in the following:



In the first level, we should traverse from left to right (order: `3`) and in the second level, we traverse from right to left (order: `4, 0`). After traversing all the nodes, the result should be `[3, 4, 0, 2, 1]`.

Note: In this exercise, the libraries `iostream`, `vector`, `stack`, `queue`, `algorithm` and `using namespace std` are used. You can write helper functions; however, you are not allowed to use other libraries.

For example:

Test	Result

`int arr[] = {0, 3, 5, 1, 2, 4};`
`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
`printVector(levelAlterTraverse(root));`
`BSTNode::deleteTree(root);`
	`[0, 3, 1, 5, 4, 2]`

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
vector<int> levelAlterTraverse(BSTNode* root) {
    vector<int> result;
    if (!root) return result;

    queue<BSTNode*> q;
    q.push(root);
    bool leftToRight = true;

    while (!q.empty()) {
        int levelSize = q.size();
        vector<int> levelNodes;

        for (int i = 0; i < levelSize; ++i) {
            BSTNode* node = q.front();
            q.pop();
            levelNodes.push_back(node->val);

            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }

        if (!leftToRight) {
            reverse(levelNodes.begin(), levelNodes.end());
        }

        result.insert(result.end(), levelNodes.begin(), levelNodes.end());
        leftToRight = !leftToRight;
    }

    return result;
}
```

	Test	Expected	Got	

✓	`int arr[] = {0, 3, 5, 1, 2, 4};`
	`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
	`printVector(levelAlterTraverse(root));`
	`BSTNode::deleteTree(root);`
		`[0, 3, 1, 5, 4, 2]`	`[0, 3, 1, 5, 4, 2]`	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **3**

Đúng

Đạt điểm 1,00 trên 1,00

Class `BSTNode` is used to store a node in binary search tree, described on the following:

```
class BSTNode {
public:
    int val;
    BSTNode *left;
    BSTNode *right;
    BSTNode() {
        this->left = this->right = nullptr;
    }
    BSTNode(int val) {
        this->val = val;
        this->left = this->right = nullptr;
    }
    BSTNode(int val, BSTNode*& left, BSTNode*& right) {
        this->val = val;
        this->left = left;
        this->right = right;
    }
};
```

Where `val` is the value of node, `left` and `right` are the pointers to the left node and right node of it, respectively. If a repeated value is inserted to the tree, it will be inserted to the left subtree.

Also, a static method named `createBSTree` is used to create the binary search tree, by iterating the argument array left-to-right and repeatedly calling `addNode` method on the root node to insert the value into the correct position. For example:

```
int arr[] = {0, 10, 20, 30};
auto root = BSTNode::createBSTree(arr, arr + 4);
```

is equivalent to

```
auto root = new BSTNode(0);
root->addNode(10);
root->addNode(20);
root->addNode(30);
```

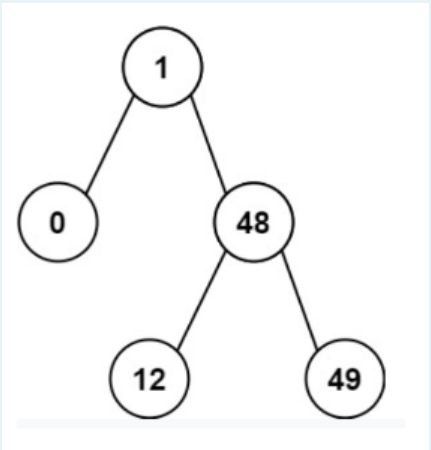
Request: Implement function:

```
int kthSmallest(BSTNode* root, int k);
```

Where `root` is the root node of given binary search tree (this tree has `n` elements) and `k` satisfy: `1 <= k <= n <= 100000`. This function returns the `k`-th smallest value in the tree.

Example:

Given a binary search tree in the following:



With `k = 2`, the result should be `1`.

Note: In this exercise, the libraries `iostream`, `vector`, `stack`, `queue`, `algorithm`, `climits` and `using namespace std` are used. You can write helper functions; however, you are not allowed to use other libraries.

For example:

Test	Result

`int arr[] = {6, 9, 2, 13, 0, 20};`
`int k = 2;`
`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
`cout << kthSmallest(root, k);`
`BSTNode::deleteTree(root);`
	2

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
int kthSmallest(BSTNode* root, int k) {
    stack<BSTNode*> stk;
    BSTNode* curr = root;
    int count = 0;

    while (!stk.empty() || curr != nullptr) {
        while (curr != nullptr) {
            stk.push(curr);
            curr = curr->left;
        }
        curr = stk.top();
        stk.pop();
        count++;
        if (count == k) return curr->val;
        curr = curr->right;
    }
    return -1; // This should never be reached if k is valid
}
```

	Test	Expected	Got	

✓	`int arr[] = {6, 9, 2, 13, 0, 20};`
	`int k = 2;`
	`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
	`cout << kthSmallest(root, k);`
	`BSTNode::deleteTree(root);`
		2	2	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **4**

Đúng

Đạt điểm 1,00 trên 1,00

Class **BTNode** is used to store a node in binary search tree, described on the following:

```
class BTNode {
    public:
        int val;
        BTNode *left;
        BTNode *right;
        BTNode() {
            this->left = this->right = NULL;
        }
        BTNode(int val) {
            this->val = val;
            this->left = this->right = NULL;
        }
        BTNode(int val, BTNode*& left, BTNode*& right) {
            this->val = val;
            this->left = left;
            this->right = right;
        }
};
```

Where `val` is the value of node (non-negative integer), `left` and `right` are the pointers to the left node and right node of it, respectively.

Also, a static method named `createBSTree` is used to create the binary search tree, by iterating the argument array left-to-right and repeatedly calling `addNode` method on the root node to insert the value into the correct position. For example:

```
int arr[] = {0, 10, 20, 30};
auto root = BSTNode::createBSTree(arr, arr + 4);
```

is equivalent to

```
auto root = new BSTNode(0);
root->addNode(10);
root->addNode(20);
root->addNode(30);
```

Request: Implement function:

```
int rangeCount(BTNode* root, int lo, int hi);
```

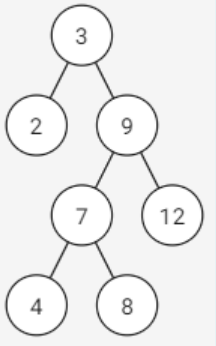
Where `root` is the root node of given binary search tree (this tree has between 0 and 100000 elements), `lo` and `hi` are 2 positives integer and `lo ≤ hi`. This function returns the number of all nodes whose values are between `[lo, hi]` in this binary search tree.

More information:

- If a node has `val` which is equal to its ancestor's, it is in the right subtree of its ancestor.

Example:

Given a binary search tree in the following:



With `lo=5, hi=10`, all the nodes satisfied are node `9, 7, 8`; there fore, the result is `3`.

Note: In this exercise, the libraries `iostream`, `stack`, `queue`, `utility` and `using namespace std` are used. You can write helper functions; however, you are not allowed to use other libraries.

For example:

Test	Result

`int value[] = {3,2,9,7,12,4,8};`
`int lo = 5, hi = 10;`
`BTNode* root = BTNode::createBSTree(value, value + sizeof(value)/sizeof(int));`
`cout << rangeCount(root, lo, hi);`
	3

`int value[] = {1167,2381,577,2568,124,1519,234,1679,2696,2359};`
`int lo = 500, hi = 2000;`
`BTNode* root = BTNode::createBSTree(value, value + sizeof(value)/sizeof(int));`
`cout << rangeCount(root, lo, hi);`
	4

Answer: (penalty regime: 0 %)

Reset answer

```
int rangeCount(BTNode* root, int lo, int hi) {
    if (root == nullptr) {
        return 0;
    }

    if (root->val < lo) {
        return rangeCount(root->right, lo, hi);
    }

    if (root->val > hi) {
        return rangeCount(root->left, lo, hi);
    }

    return 1 + rangeCount(root->left, lo, hi) + rangeCount(root->right, lo, hi);
}
```

	Test	Expected	Got	

✓	int value[] = {3,2,9,7,12,4,8};
	int lo = 5, hi = 10;
	BTNode* root = BTNode::createBSTree(value, value + sizeof(value)/sizeof(int));
	cout << rangeCount(root, lo, hi);
		3	3	✓

✓	int value[] = {1167,2381,577,2568,124,1519,234,1679,2696,2359};
	int lo = 500, hi = 2000;
	BTNode* root = BTNode::createBSTree(value, value + sizeof(value)/sizeof(int));
	cout << rangeCount(root, lo, hi);
		4	4	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **5**

Đúng

Đạt điểm 1,00 trên 1,00

Class `BSTNode` is used to store a node in binary search tree, described on the following:

```
class BSTNode {
public:
    int val;
    BSTNode *left;
    BSTNode *right;
    BSTNode() {
        this->left = this->right = nullptr;
    }
    BSTNode(int val) {
        this->val = val;
        this->left = this->right = nullptr;
    }
    BSTNode(int val, BSTNode*& left, BSTNode*& right) {
        this->val = val;
        this->left = left;
        this->right = right;
    }
};
```

Where `val` is the value of node, `left` and `right` are the pointers to the left node and right node of it, respectively. If a repeated value is inserted to the tree, it will be inserted to the left subtree.

Also, a static method named `createBSTree` is used to create the binary search tree, by iterating the argument array left-to-right and repeatedly calling `addNode` method on the root node to insert the value into the correct position. For example:

```
int arr[] = {0, 10, 20, 30};
auto root = BSTNode::createBSTree(arr, arr + 4);
```

is equivalent to

```
auto root = new BSTNode(0);
root->addNode(10);
root->addNode(20);
root->addNode(30);
```

Request: Implement function:

```
int singleChild(BSTNode* root);
```

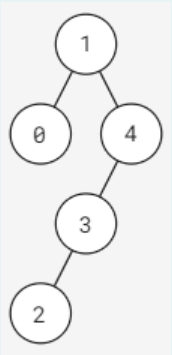
Where `root` is the root node of given binary search tree (this tree has between 0 and 100000 elements). This function returns the number of single children in the tree.

More information:

- A node is called a **single child** if its parent has only one child.

Example:

Given a binary search tree in the following:



There are 2 single children: node 2 and node 3.

Note: In this exercise, the libraries `iostream` and `using namespace std` are used. You can write helper functions; however, you are not allowed to use other libraries.

For example:

Test	Result

`int arr[] = {0, 3, 5, 1, 2, 4};`
`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
`cout << singleChild(root);`
`BSTNode::deleteTree(root);`
	3

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
int singleChild(BSTNode* root) {
    if (!root) return 0;

    int count = 0;
    if ((root->left && !root->right) || (!root->left && root->right)) {
        count = 1;
    }

    return count + singleChild(root->left) + singleChild(root->right);
}
```

	Test	Expected	Got	

✓	`int arr[] = {0, 3, 5, 1, 2, 4};`
	`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
	`cout << singleChild(root);`
	`BSTNode::deleteTree(root);`
		3	3	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **6**

Đúng

Đạt điểm 1,00 trên 1,00

Class `BSTNode` is used to store a node in binary search tree, described on the following:

```
class BSTNode {
public:
    int val;
    BSTNode *left;
    BSTNode *right;
    BSTNode() {
        this->left = this->right = nullptr;
    }
    BSTNode(int val) {
        this->val = val;
        this->left = this->right = nullptr;
    }
    BSTNode(int val, BSTNode*& left, BSTNode*& right) {
        this->val = val;
        this->left = left;
        this->right = right;
    }
};
```

Where `val` is the value of node, `left` and `right` are the pointers to the left node and right node of it, respectively. If a repeated value is inserted to the tree, it will be inserted to the left subtree.

Also, a static method named `createBSTree` is used to create the binary search tree, by iterating the argument array left-to-right and repeatedly calling `addNode` method on the root node to insert the value into the correct position. For example:

```
int arr[] = {0, 10, 20, 30};
auto root = BSTNode::createBSTree(arr, arr + 4);
```

is equivalent to

```
auto root = new BSTNode(0);
root->addNode(10);
root->addNode(20);
root->addNode(30);
```

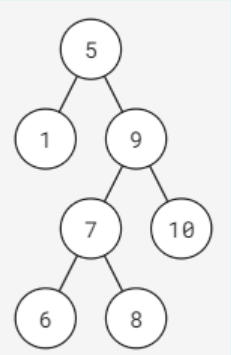
Request: Implement function:

```
BSTNode* subtreeWithRange(BSTNode* root, int lo, int hi);
```

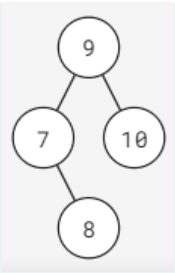
Where `root` is the root node of given binary search tree (this tree has between 0 and 100000 elements). This function returns the binary search tree after deleting all nodes whose values are outside the range `[lo, hi]` (inclusive).

Example:

Given a binary search tree in the following:



With `lo = 7` and `hi = 10`, the result should be:



Note: In this exercise, the libraries `iostream` and `using namespace std` are used. You can write helper functions; however, you are not allowed to use other libraries.

For example:

Test	Result

`int arr[] = {0, 3, 5, 1, 2, 4};`
`int lo = 1, hi = 3;`
`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
`root = subtreeWithRange(root, lo, hi);`
`BSTNode::printPreorder(root);`
`BSTNode::deleteTree(root);`
	3 1 2

Answer: (penalty regime: 0, 0, 0, 5, 10, ... %)

Reset answer

```
BSTNode* subtreeWithRange(BSTNode* root, int lo, int hi) {
    if (!root) return nullptr;

    // Trim the left and right subtrees
    root->left = subtreeWithRange(root->left, lo, hi);
    root->right = subtreeWithRange(root->right, lo, hi);

    // Now deal with the current node
    if (root->val < lo) {
        // Current node is too small, discard it and return the trimmed right subtree
        BSTNode* rightChild = root->right;
        delete root; // Free the memory of the current node
        return rightChild;
    }
    if (root->val > hi) {
        // Current node is too large, discard it and return the trimmed left subtree
        BSTNode* leftChild = root->left;
        delete root; // Free the memory of the current node
        return leftChild;
    }

    // Current node is within the range, keep it
    return root;
}
```

	Test	Expected	Got	

✓	`int arr[] = {0, 3, 5, 1, 2, 4};`
	`int lo = 1, hi = 3;`
	`BSTNode* root = BSTNode::createBSTree(arr, arr + sizeof(arr)/sizeof(int));`
	`root = subtreeWithRange(root, lo, hi);`
	`BSTNode::printPreorder(root);`
	`BSTNode::deleteTree(root);`
		3 1 2	3 1 2	✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **7**

Đúng

Đạt điểm 1,00 trên 1,00

Given class **BinarySearchTree**, you need to finish method **find(i)** to check whether value i is in the tree or not; method **sum(l,r)** to calculate sum of all all elements v in the tree that has value greater than or equal to l and less than or equal to r.

```
#include <iostream>
#include <string>
#include <sstream>

using namespace std;

template<class T>
class BinarySearchTree
{
public:
    class Node;

private:
    Node* root;

public:
    BinarySearchTree() : root(nullptr) {}
    ~BinarySearchTree()
    {
        // You have to delete all Nodes in BinaryTree. However in this task, you can ignore it.
    }

    class Node
    {
    private:
        T value;
        Node* pLeft, * pRight;
        friend class BinarySearchTree<T>;

    public:
        Node(T value) : value(value), pLeft(NULL), pRight(NULL) {}
        ~Node() {}
    };
    Node* addRec(Node* root, T value);
    void add(T value) ;
    // STUDENT ANSWER BEGIN

    // STUDENT ANSWER END
};
```

For example:

Test	Result

`BinarySearchTree<int> bst;`
`for (int i = 0; i < 10; ++i) {`
`  bst.add(i);`
`}`
`cout << bst.find(7) << endl;`
`cout << bst.sum(0, 4) << endl`
	1
	10

Answer: (penalty regime: 5, 10, 15, ... %)

Reset answer

```
1 // STUDENT ANSWER BEGIN
2 bool findRec(Node* root, T i) const {
```

```
    bool findRec(Node* root, T i) const {
        if (root == nullptr) {
            return false;
        }
        if (root->value == i) {
            return true;
        }
        if (i < root->value) {
            return findRec(root->pLeft, i);
        }
        return findRec(root->pRight, i);
    }

    bool find(T i) const {
        return findRec(root, i);
    }

    T sumRec(Node* root, T l, T r) const {
        if (root == nullptr) {
            return 0;
        }
        if (root->value < l) {
            return sumRec(root->pRight, l, r);
        }
        if (root->value > r) {
            return sumRec(root->pLeft, l, r);
        }
        return root->value + sumRec(root->pLeft, l, r) + sumRec(root->pRight, l, r);
    }

    T sum(T l, T r) const {
        return sumRec(root, l, r);
    }
    // STUDENT ANSWER END
```

	Test	Expected	Got	

✓	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(i);`
	`}`
	`cout << bst.find(7) << endl;`
	`cout << bst.sum(0, 4) << endl`
		1
		10
			1
			10
				✓

✓	`int values[] = { 66,60,84,67,21,45,62,1,80,35 };`
	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(values[i]);`
	`}`
	
	`cout << bst.find(5) << endl;`
	`cout << bst.sum(10, 40);`
		0
		56
			0
			56
				✓

✓	`int values[] = { 38,0,98,38,99,67,19,70,55,6 };`
	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(values[i]);`
	`}`
	
	`cout << bst.find(5) << endl;`
	`cout << bst.sum(10, 40);`
		0
		95
			0
			95
				✓

	Test	Expected	Got	

✓	int values[] = { 34,81,73,48,66,91,19,84,78,79 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 10; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(5) << endl;
	cout << bst.sum(10, 40);
		0
		53
			0
			53
				✓

✓	int values[] = { 94,61,75,36,34,58,62,74,54,90 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 10; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(34) << endl;
	cout << bst.sum(10, 40);
		1
		70
			1
			70
				✓

✓	int values[] = { 32,0,2,84,34,78,70,60,95,71,26,62,0,22,95 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(34) << endl;
	cout << bst.sum(10, 40);
		1
		114
			1
			114
				✓

✓	int values[] = { 53,24,32,40,80,47,81,88,42,29,31,91,77,73,90 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(34) << endl;
	cout << bst.sum(10, 40);
		0
		156
			0
			156
				✓

✓	int values[] = { 32,19,23,33,76,1,37,53,18,89,28,1,77,52,17 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(34) << endl;
	cout << bst.sum(10, 40);
		0
		207
			0
			207
				✓

✓	int values[] = { 25,29,57,30,62,56,60,55,88,56,70,83,56,75,17 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(34) << endl;
	cout << bst.sum(10, 40);
		0
		101
			0
			101
				✓

✓	int values[] = { 75,13,83,83,30,40,10,86,17,21,45,22,22,72,63 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.find(34) << endl;
	cout << bst.sum(10, 40);
		0
		175
			0
			175
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.

Câu hỏi **8**

Đúng

Đạt điểm 1,00 trên 1,00

Given class **BinarySearchTree**, you need to finish method getMin() and getMax() in this question.

```
#include <iostream>
#include <string>
#include <sstream>

using namespace std;

template<class T>
class BinarySearchTree
{
public:
    class Node;

private:
    Node* root;

public:
    BinarySearchTree() : root(nullptr) {}
    ~BinarySearchTree()
    {
        // You have to delete all Nodes in BinaryTree. However in this task, you can ignore it.
    }

    class Node
    {
    private:
        T value;
        Node* pLeft, * pRight;
        friend class BinarySearchTree<T>;

    public:
        Node(T value) : value(value), pLeft(NULL), pRight(NULL) {}
        ~Node() {}
    };
    Node* addRec(Node* root, T value);
    void add(T value) ;
    // STUDENT ANSWER BEGIN

    // STUDENT ANSWER END
};
```

For example:

Test	Result

BinarySearchTree<int> bst;
for (int i = 0; i < 10; ++i) {
bst.add(i);
}
cout << bst.getMin() << endl;
cout << bst.getMax() << endl;
	0
	9

Answer: (penalty regime: 5, 10, 15, ... %)

Reset answer

```
// STUDENT ANSWER BEGIN
    // You can define other functions here to help you.

    T getMin() {
        if (!root) throw runtime_error("Tree is empty");
        Node* current = root;
        while (current->pLeft) {
            current = current->pLeft;
        }
        return current->value;
    }

    T getMax() {
        if (!root) throw runtime_error("Tree is empty");
        Node* current = root;
        while (current->pRight) {
            current = current->pRight;
        }
        return current->value;
    }
    // STUDENT ANSWER END
```

	Test	Expected	Got	

✓	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(i);`
	`}`
	`cout << bst.getMin() << endl;`
	`cout << bst.getMax() << endl;`
		0
		9
			0
			9
				✓

✓	`int values[] = { 66,60,84,67,21,45,62,1,80,35 };`
	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(values[i]);`
	`}`
	
	`cout << bst.getMin() << endl;`
	`cout << bst.getMax() << endl;`
		1
		84
			1
			84
				✓

✓	`int values[] = { 38,0,98,38,99,67,19,70,55,6 };`
	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(values[i]);`
	`}`
	
	`cout << bst.getMin() << endl;`
	`cout << bst.getMax() << endl;`
		0
		99
			0
			99
				✓

✓	`int values[] = { 34,81,73,48,66,91,19,84,78,79 };`
	`BinarySearchTree<int> bst;`
	`for (int i = 0; i < 10; ++i) {`
	`  bst.add(values[i]);`
	`}`
	
	`cout << bst.getMin() << endl;`
	`cout << bst.getMax() << endl;`
		19
		91
			19
			91
				✓

	Test	Expected	Got	

✓	int values[] = { 94,61,75,36,34,58,62,74,54,90 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 10; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.getMin() << endl;
	cout << bst.getMax() << endl;
		34
		94
			34
			94
				✓

✓	int values[] = { 32,0,2,84,34,78,70,60,95,71,26,62,0,22,95 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.getMin() << endl;
	cout << bst.getMax() << endl;
		0
		95
			0
			95
				✓

✓	int values[] = { 53,24,32,40,80,47,81,88,42,29,31,91,77,73,90 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.getMin() << endl;
	cout << bst.getMax() << endl;
		24
		91
			24
			91
				✓

✓	int values[] = { 32,19,23,33,76,1,37,53,18,89,28,1,77,52,17 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.getMin() << endl;
	cout << bst.getMax() << endl;
		1
		89
			1
			89
				✓

✓	int values[] = { 25,29,57,30,62,56,60,55,88,56,70,83,56,75,17 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.getMin() << endl;
	cout << bst.getMax() << endl;
		17
		88
			17
			88
				✓

✓	int values[] = { 75,13,83,83,30,40,10,86,17,21,45,22,22,72,63 };
	BinarySearchTree<int> bst;
	for (int i = 0; i < 15; ++i) {
	bst.add(values[i]);
	}
	
	cout << bst.getMin() << endl;
	cout << bst.getMax() << endl;
		10
		86
			10
			86
				✓

Passed all tests! ✓

Đúng

Marks for this submission: 1,00/1,00.